

Emergent-UI Onboarding Architecture

Ionirix Platform — Runnable VS Code Workspace Plan

Version: 1.0.0

Date: April 2026

Author: Mirnes — Solo Founder-Architect

Category: Technical Architecture Blueprint / Workspace Plan

Document Scope

This document defines the complete workspace structure, component architecture, state machine configuration, JSON-driven layout schema, event-driven reflow engine, and phased implementation plan for the Ionirix Emergent-UI onboarding system. Every section is implementation-ready for a solo founder working in VS Code with React 19, TypeScript 5.x, and XState v5.

1. Architecture Overview

1.1 Emergent-UI Philosophy

Emergent-UI is the core architectural principle of the Ionirix platform. Unlike conventional UI frameworks that prescribe fixed layouts and hardcoded navigation paths, Emergent-UI surfaces **discover their own optimal layout** through real-time analysis of user interaction signals, viewport constraints, and behavioral context. The UI is not designed — it is *grown*.

The onboarding flow is the first surface where this philosophy manifests: rather than a rigid step-by-step wizard, the onboarding adapts its structure, density, and pacing based on how the user interacts — skipping steps for power users, expanding

guidance for cautious ones, and recomposing its spatial layout as the user reveals their intent.

1.2 Core Principles

Principle	Definition	Implementation
Spatial Intelligence	UI zones are aware of their neighbors, overlap, visual weight, and viewport budget	SpatialAnalyzer monitors zone relationships; feeds ReflowEngine
Adaptive Reflow	Layout recalculates in response to interaction signals, not just viewport changes	ReflowEngine resolves JSON schema against current constraints on every trigger
Behavioral Emergence	UI behaviors are registered, not hardcoded; new behaviors compose with existing ones	BehaviorRegistry with pluggable behavior modules evaluated per frame/event
Declarative Surfaces	All layout and behavior configuration is JSON-driven and schema-validated	LayoutSchema JSON files define zones, grids, behaviors, and transitions
Sovereign Aesthetic	Cold premium palette, glass-morphism, cinematic depth — Ionirix brand DNA	Design tokens system with ion-blue, spectral-cyan, quantum-white, amber, pine-black

1.3 System Architecture Diagram

The Emergent-UI stack is organized as a six-layer pipeline. Each layer communicates through the EventBus. Data flows downward for rendering; signals flow upward for adaptation.

Layer	Component	Responsibility	Communicates With
L1 — Client	React Components	Render surfaces, capture interactions, emit	L2, L5

Layer	Component	Responsibility	Communicates With
		events	
L2 — Layout Engine	ReflowEngine, LayoutResolver, SpatialAnalyzer	Resolve JSON schemas to concrete layouts, detect spatial conflicts	L1, L3, L4
L3 — State Machine	OnboardingMachine, UIMachine (XState v5)	Manage flow state, UI mode, guard transitions	L2, L4, L5
L4 — Behavior Registry	BehaviorRegistry, behavior modules	Evaluate and execute registered behaviors against current context	L2, L3, L5
L5 — Event Bus	EventBus (typed pub/sub)	Route typed events between all layers with discriminated unions	All layers
L6 — Persistence	LocalStorage / IndexedDB adapter	Persist user preferences, interaction history, layout state	L3, L5

Signal flow: User interaction → L1 emits `INTERACTION` event → L5 routes to L4 → L4 evaluates behaviors → L4 emits `REFLOW_REQUEST` → L2 resolves layout → L2 emits `LAYOUT_CHANGE` → L1 re-renders → L3 records state transition → L6 persists.

2. VS Code Workspace Configuration

2.1 Workspace File

Save as `ionirix-emergent-ui.code-workspace` at the repository root.

```
{
  "folders": [
    { "name": "📁 Root", "path": "." },
    { "name": "⚙️ Core Engine", "path": "src/core/engine" },
    { "name": "🔄 State Machines", "path": "src/core/state" },
    { "name": "📋 Registry", "path": "src/core/registry" },
    { "name": "📡 Events", "path": "src/core/events" }
  ]
}
```

```
{ "src/core/events" }, { "name": "🗺️ Schema", "path": "src/core/schema" },  
{ "name": "🧩 Components", "path": "src/components" }, { "name": "🔥 Hooks", "path": "src/hooks" }, { "name": "✏️ Types", "path": "src/types" }, { "name": "🎨 Styles", "path": "src/styles" }, { "name": "🪄 Tests", "path": "tests" }, { "name": "📖 Docs", "path": "docs" } ], "settings": { "typescript.tsdk": "node_modules/typescript/lib", "typescript.preferences.importModuleSpecifier": "non-relative", "editor.formatOnSave": true, "editor.defaultFormatter": "esbenp.prettier-vscode", "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit", "source.organizeImports": "explicit" }, "editor.tabSize": 2, "typescript.preferences.quoteStyle": "single", "tailwindCSS.experimental.classRegex": [ ["cva\\(([^\\)]*)\\)", "[\\\"'`"] ([^\\\"'`]*)\\.??([\\\"'`]") ] ], "files.associations": { "**.schema.json": "json" }, "search.exclude": { "**/node_modules": true, "**/dist": true, "**/.turbo": true } }, "extensions": { "recommendations": [ "statelyai.stately-vsc", "bradlc.vsc-tailwindcss", "dbaeumer.vsc-eslint", "esbenp.prettier-vscode", "rbbbit.typescript-hero", "ms-vscode.vsc-typescript-next", "vitest.explorer", "streetsidesoftware.code-spell-checker" ] }, "tasks": { "version": "2.0.0", "tasks": [ { "label": "dev", "type": "shell", "command": "npx vite --host", "group": { "kind": "build", "isDefault": true }, "problemMatcher": [], "presentation": { "reveal": "always", "panel": "dedicated" } }, { "label": "build", "type": "shell", "command": "npx tsc --noEmit && npx vite build", "group": "build", "problemMatcher": ["$tsc"], "type": "shell", "command": "npx vitest run", "group": "test", "problemMatcher": [] }, { "label": "test:watch", "type": "shell", "command": "npx vitest", "group": "test", "isBackground": true, "problemMatcher": [] }, { "label": "lint", "type": "shell", "command": "npx eslint src/ --ext .ts,.tsx --fix", "problemMatcher": ["$eslint-stylish"], "type": "shell", "command": "npx tsc --noEmit --watch", "isBackground": true, "problemMatcher": ["$tsc-watch"] } ] }
```

2.2 Recommended Extensions

Extension	ID	Purpose
Stately XState	statelyai.stately-vscode	Visual state machine editor, XState v5 IntelliSense
Tailwind CSS IntelliSense	bradlc.vscode-tailwindcss	Class autocomplete, hover preview, linting
ESLint	dbaeumer.vscode-eslint	Lint-on-save with auto-fix
Prettier	esbenp.prettier-vscode	Format-on-save, consistent code style

Extension	ID	Purpose
TypeScript Hero	rbbbit.typescript-hero	Auto-organize imports, quick-fix missing imports
Vitest Explorer	vitest.explorer	Run/debug tests from sidebar

3. Folder Structure

Complete directory tree for the `ionirix-emergent-ui/` monorepo. Every file listed below is referenced in subsequent sections.

```

ionirix-emergent-ui/ | .vscode/ | settings.json |
extensions.json | tasks.json | src/ | core/ | engine/
| ReflowEngine.ts | LayoutResolver.ts |
| SpatialAnalyzer.ts | index.ts | state/
| OnboardingMachine.ts | UIStateMachine.ts |
guards.ts | actions.ts | index.ts |
registry/ | BehaviorRegistry.ts |
ComponentRegistry.ts | behaviors/ |
AdaptiveReveal.ts | SpatialCollapse.ts |
ContextualElevation.ts | ProgressiveDisclosure.ts |
| index.ts | index.ts | events/ |
EventBus.ts | EventTypes.ts | handlers/ |
| InteractionHandler.ts | NavigationHandler.ts |
| ReflowHandler.ts | index.ts |
index.ts | schema/ | LayoutSchema.ts |
validators.ts | defaults/ |
onboarding.schema.json | dashboard.schema.json |
| editorial.schema.json | index.ts | components/ |
| onboarding/ | OnboardingShell.tsx |
StepRenderer.tsx | WelcomePane.tsx |
CapabilitySelector.tsx | EnvironmentConfigurator.tsx |
| FlowSummary.tsx | index.ts | surfaces/ |
| AdaptiveSurface.tsx | GlassCard.tsx |
FloatingPanel.tsx | ContextRibbon.tsx | index.ts
| primitives/ | IonButton.tsx |
IonInput.tsx | IonToggle.tsx | IonProgress.tsx |
| IonTooltip.tsx | index.ts | layout/ |
| EmergentGrid.tsx | ReflowContainer.tsx |
ZoneManager.tsx | index.ts | hooks/ |
useReflow.ts | useBehavior.ts | useOnboardingState.ts |
| useLayoutSchema.ts | useSpatialContext.ts |
index.ts | types/ | layout.types.ts |
behavior.types.ts | onboarding.types.ts | events.types.ts
| schema.types.ts | index.ts | utils/ |
spatial.ts | animation.ts | persistence.ts |
index.ts | styles/ | tokens.css | glass.css |
| animations.css | emergent.css | App.tsx | schemas/ |
| layout.schema.json | behavior.schema.json | tests/ |
engine/ | ReflowEngine.test.ts | LayoutResolver.test.ts |
| SpatialAnalyzer.test.ts | state/ |
OnboardingMachine.test.ts | UIStateMachine.test.ts |

```

```

registry/ | | | BehaviorRegistry.test.ts | | | behaviors.test.ts
| | | components/ | | | OnboardingShell.test.tsx | | |
StepRenderer.test.tsx | docs/ | | ARCHITECTURE.md | | |
BEHAVIOR_CATALOG.md | | | SCHEMA_REFERENCE.md | package.json | |
tsconfig.json | | | tailwind.config.ts | vite.config.ts | | README.md

```

4. Component Definitions

4.1 Core Engine Components

4.1.1 ReflowEngine

Orchestrates layout recalculation when UI signals trigger reflow. Accepts a LayoutSchema, current viewport context, and user interaction history. Outputs a resolved layout map. This is the central coordinator — it does not render, it computes.

```

// src/core/engine/ReflowEngine.ts import type { LayoutSchema,
ResolvedLayout, ReflowTrigger, ViewportContext, SpatialWeightMap,
CollisionReport, } from '@types'; import type { Behavior } from
'@types/behavior.types'; export interface ReflowEngineConfig { /**
Maximum reflow cycles before force-commit (prevents infinite loops) */
maxIterations: number; /** Debounce interval in ms for rapid-fire triggers
*/ debounceMs: number; /** Enable spatial collision detection */
spatialAwareness: boolean; } export interface ReflowResult { layout:
ResolvedLayout; mutations: LayoutMutation[]; collisions:
CollisionReport[]; iterationCount: number; timestamp: number; } export
interface LayoutMutation { zoneId: string; property: string;
previousValue: string | number; nextValue: string | number; reason:
ReflowTrigger; } export class ReflowEngine { private schema: LayoutSchema
| null = null; private config: ReflowEngineConfig; private currentLayout:
ResolvedLayout | null = null; private viewport: ViewportContext;
constructor(config: Partial<ReflowEngineConfig> = {}) { /* ... */ /**
Load and validate a layout schema */ initialize(schema: LayoutSchema): void
{ /* ... */ /** Process a reflow trigger and return a new resolved
layout */ requestReflow(trigger: ReflowTrigger): ReflowResult { /* ... */ }
/** Apply registered behaviors to a resolved layout */
applyBehaviors( layout: ResolvedLayout, behaviors: Behavior[] ):
ResolvedLayout { /* ... */ /** Commit a resolved layout – emits
LAYOUT_CHANGE event */ commit(layout: ResolvedLayout): void { /* ... */ }
/** Update viewport context (called on resize / orientation change) */
setViewport(ctx: ViewportContext): void { /* ... */ // --- Private ---
private _resolveConstraints( zones: ZoneDefinition[], viewport:
ViewportContext ): ConstraintResult[] { /* ... */ private
_calculateSpatialWeights( zones: ZoneDefinition[] ): SpatialWeightMap {
/* ... */ private _detectCollisions( layout: ResolvedLayout ):
CollisionReport[] { /* ... */ private _iterateUntilStable( layout:
ResolvedLayout, maxIterations: number ): ResolvedLayout { /* ... */ } }

```

4.1.2 LayoutResolver

Takes a JSON layout schema and resolves it against current constraints (viewport, user preferences, active behaviors). Returns concrete CSS grid definitions that the rendering layer consumes directly.

```
// src/core/engine/LayoutResolver.ts import type { LayoutSchema,
ResolvedLayout, ViewportContext, UserPreferences, ZoneDefinition,
GridDefinition, } from '@types'; export interface ResolvedZone { id:
string; component: string; gridArea: string; computedWidth: number;
computedHeight: number; visibility: 'visible' | 'collapsed' | 'hidden' |
'merged'; mergedInto?: string; priority: number; zIndex: number; }
export interface ResolvedGrid { columns: string; rows: string; gap:
string; areas: string[]; totalWidth: number; totalHeight: number; }
export class LayoutResolver { /** Resolve a schema to concrete layout
definitions */ resolve( schema: LayoutSchema, viewport:
ViewportContext, preferences: UserPreferences, activeBehaviors:
string[] ): ResolvedLayout { /* ... */ } /** Resolve a single zone
against constraints */ resolveZone( zone: ZoneDefinition, viewport:
ViewportContext, siblingZones: ZoneDefinition[] ): ResolvedZone {
/* ... */ } /** Build CSS grid template from schema grid definition */
resolveGrid( grid: GridDefinition, viewport: ViewportContext ):
ResolvedGrid { /* ... */ } /** Determine which zones should collapse at
current viewport */ evaluateCollapseRules( zones: ZoneDefinition[],
viewportWidth: number ): Map<string, 'hide' | 'minimize' | 'stack' |
'merge'> { /* ... */ } }
```

4.1.3 SpatialAnalyzer

Monitors spatial relationships between UI zones. Detects overlap, proximity, and visual weight distribution. Feeds signals to the ReflowEngine when spatial conditions change.

```
// src/core/engine/SpatialAnalyzer.ts export interface SpatialRelation
{ zoneA: string; zoneB: string; distance: number; overlap: boolean;
overlapArea: number; relativePosition: 'above' | 'below' | 'left' | 'right'
| 'overlapping'; } export interface VisualWeightReport { zoneId: string;
weight: number; // 0-1 normalized contentDensity: number;
interactionFrequency: number; spatialImportance: number; } export
interface SpatialSnapshot { timestamp: number; relations:
SpatialRelation[]; weights: VisualWeightReport[]; collisions:
Array<{ zoneA: string; zoneB: string; severity: number }>; balance:
{ horizontal: number; vertical: number }; // -1 to 1 } export class
SpatialAnalyzer { /** Analyze current DOM positions of all registered zones
*/ analyze(zoneElements: Map<string, DOMRect>): SpatialSnapshot { /* ... */
} /** Check if two zones overlap */ detectOverlap(a: DOMRect, b:
DOMRect): boolean { /* ... */ } /** Calculate Euclidean distance between
zone centers */ calculateDistance(a: DOMRect, b: DOMRect): number { /* ...
*/ } /** Compute visual weight based on size, content, and interaction
data */ computeVisualWeight( zone: DOMRect, contentDensity: number,
interactionCount: number ): number { /* ... */ } /** Detect visual
balance across the viewport */ assessBalance( zones: Map<string,
DOMRect> ): { horizontal: number; vertical: number } { /* ... */ } }
```

4.2 State Machine (XState v5)

4.2.1 OnboardingMachine — Full Configuration

Primary state machine for the onboarding flow. Uses XState v5 `setup()` API with typed context, events, guards, and actions.

```
// src/core/state/OnboardingMachine.ts import { setup, assign, fromPromise }
from 'xstate'; // — Context Shape

export interface
OnboardingContext { currentStep: number; userProfile: { name: string;
role: string; experience: 'beginner' | 'intermediate' | 'advanced' |
null; }; selectedCapabilities: string[]; environmentConfig:
{ theme: 'dark' | 'light' | 'system'; density: 'compact' |
'comfortable' | 'spacious'; motionPreference: 'full' | 'reduced' |
'none'; }; spatialPreferences: { layoutMode: 'grid' | 'stack' |
'float'; sidebarPosition: 'left' | 'right' | 'hidden'; zoneCount:
number; }; interactionHistory: Array<{ event: string; timestamp:
number; metadata?: Record<string, unknown>; }>; } // — Event Types
type OnboardingEvent = |
{ type: 'NEXT' } | { type: 'BACK' } | { type: 'SKIP' } | { type:
'UPDATE_PROFILE'; data: Partial<OnboardingContext['userProfile']> } |
{ type: 'SELECT_CAPABILITY'; capability: string } | { type:
'REMOVE_CAPABILITY'; capability: string } | { type: 'CONFIGURE_ENV';
config: Partial<OnboardingContext['environmentConfig']> } | { type:
'SET_SPATIAL'; prefs: Partial<OnboardingContext['spatialPreferences']> } |
{ type: 'CALIBRATE' } | { type: 'COMPLETE' }; // — Machine Definition
export const onboardingMachine =
setup({ types: { context: {} as OnboardingContext, events: {} as
OnboardingEvent, }, guards: { hasProfile: ({ context }) =>
context.userProfile.name.length > 0 && context.userProfile.role.length
> 0 && context.userProfile.experience !== null, hasCapabilities:
({ context }) => context.selectedCapabilities.length >= 1,
isCalibrated: ({ context }) => context.spatialPreferences.zoneCount > 0
&& context.spatialPreferences.layoutMode !== null, canGoBack: ({ context
}) => context.currentStep > 0, }, actions: { incrementStep:
assign({ currentStep: ({ context }) => context.currentStep + 1, }),
decrementStep: assign({ currentStep: ({ context }) => Math.max(0,
context.currentStep - 1, }), updateProfile:
assign({ userProfile: ({ context, event }) => { if
(event.type !== 'UPDATE_PROFILE') return context.userProfile; return
{ ...context.userProfile, ...event.data; }, }, },
addCapability: assign({ selectedCapabilities: ({ context, event }) => {
if (event.type !== 'SELECT_CAPABILITY') return context.selectedCapabilities;
if (context.selectedCapabilities.includes(event.capability))
{ return context.selectedCapabilities; } return
[...context.selectedCapabilities, event.capability]; }, }, },
removeCapability: assign({ selectedCapabilities: ({ context, event })
=> { if (event.type !== 'REMOVE_CAPABILITY') return
context.selectedCapabilities; return
context.selectedCapabilities.filter(c => c !== event.capability); }, }, },
setEnvironment: assign({ environmentConfig: ({ context,
event }) => { if (event.type !== 'CONFIGURE_ENV') return
context.environmentConfig; return
{ ...context.environmentConfig, ...event.config; }, }, }, },
setSpatial: assign({ spatialPreferences: ({ context, event }) => {
if (event.type !== 'SET_SPATIAL') return context.spatialPreferences;
```

```

return { ...context.spatialPreferences, ...event.prefs }; }, },
recordInteraction: assign({ interactionHistory: ({ context, event }) =>
[
...context.interactionHistory, { event: event.type,
timestamp: Date.now() }, ], }, persistState: ({ context }) =>
{
localStorage.setItem('ionirix:onboarding',
JSON.stringify(context) ); }, }, }).createMachine({ id:
'onboarding', initial: 'idle', context: { currentStep: 0,
userProfile: { name: '', role: '', experience: null },
selectedCapabilities: [], environmentConfig: { theme: 'dark',
density: 'comfortable', motionPreference: 'full', },
spatialPreferences: { layoutMode: 'grid', sidebarPosition:
'left', zoneCount: 0, }, interactionHistory: [], },
states: { idle: { on: { NEXT: { target: 'welcome', actions:
['recordInteraction'] } }, }, welcome: { entry:
['incrementStep'], on: { NEXT: { target: 'profiling', actions:
['recordInteraction'] }, SKIP: { target: 'capabilitySelection',
actions: ['recordInteraction'] }, }, profiling:
{ entry: ['incrementStep'], on: { UPDATE_PROFILE:
{ actions: ['updateProfile', 'recordInteraction'] }, NEXT: {
target: 'capabilitySelection', guard: 'hasProfile',
actions: ['recordInteraction', 'persistState'], }, BACK: {
target: 'welcome', guard: 'canGoBack', actions:
['decrementStep', 'recordInteraction'], }, }, },
capabilitySelection: { entry: ['incrementStep'], on:
{ SELECT_CAPABILITY: { actions: ['addCapability',
'recordInteraction'] }, REMOVE_CAPABILITY: { actions:
['removeCapability', 'recordInteraction'] }, NEXT:
{ target: 'environmentSetup', guard: 'hasCapabilities',
actions: ['recordInteraction', 'persistState'], }, BACK: {
target: 'profiling', actions: ['decrementStep',
'recordInteraction'], }, }, }, environmentSetup: {
entry: ['incrementStep'], on: { CONFIGURE_ENV: { actions:
['setEnvironment', 'recordInteraction'] }, NEXT: { target:
'spatialCalibration', actions: ['recordInteraction',
'persistState'], }, BACK: { target:
'capabilitySelection', actions: ['decrementStep',
'recordInteraction'], }, }, }, spatialCalibration: {
entry: ['incrementStep'], on: { SET_SPATIAL: { actions:
['setSpatial', 'recordInteraction'] }, CALIBRATE: { actions:
['recordInteraction'], NEXT: { target: 'summary',
guard: 'isCalibrated', actions: ['recordInteraction',
'persistState'], }, BACK: { target:
'environmentSetup', actions: ['decrementStep',
'recordInteraction'], }, }, }, summary: { entry:
['incrementStep'], on: { COMPLETE: { target:
'complete', actions: ['recordInteraction', 'persistState'],
}, BACK: { target: 'spatialCalibration', actions:
['decrementStep', 'recordInteraction'], }, }, },
complete: { type: 'final', entry:
['persistState'], }, }, });

```

4.2.2 UIStateMachine

Controls emergent UI modes with parallel state regions for independent layout concerns.

```

// src/core/state/UIStateMachine.ts import { setup, assign } from 'xstate';
export interface UIContext { activeZones: string[]; focusedZone: string |

```

```

null; densityLevel: number; // 0-100 immersionDepth: number; // 0-3 }
type UIEvent = | { type: 'FOCUS_ZONE'; zoneId: string } | { type:
'BLUR_ZONE' } | { type: 'EXPAND' } | { type: 'COLLAPSE' } | { type:
'ENTER_IMMERSIVE' } | { type: 'EXIT_IMMERSIVE' } | { type: 'SET_DENSITY';
level: number } | { type: 'RESET' }; export const uiStateMachine = setup({
types: { context: {} as UIContext, events: {} as
UIEvent, }, }).createMachine({ id: 'ui', type: 'parallel', context: {
activeZones: [], focusedZone: null, densityLevel: 50,
immersionDepth: 0, }, states: { mode: { initial: 'default',
states: { default: { on: { FOCUS_ZONE:
{ target: 'focused' }, ENTER_IMMERSIVE: { target: 'immersive' },
}, }, focused: { on: { BLUR_ZONE:
{ target: 'default' }, EXPAND: { target: 'expanded' },
ENTER_IMMERSIVE: { target: 'immersive' }, }, },
expanded: { on: { COLLAPSE: { target: 'focused' },
BLUR_ZONE: { target: 'default' }, }, }, immersive:
{ on: { EXIT_IMMERSIVE: { target: 'default' },
}, }, minimal: { on: { RESET: { target:
'default' }, }, }, }, density:
{ initial: 'comfortable', states: { compact:
{ on: { SET_DENSITY: { target: 'comfortable' } }, },
comfortable: { on: { SET_DENSITY:
[ { target: 'compact', guard: ({ event }) => event.level <
33 }, { target: 'spacious', guard: ({ event }) => event.level >
66 }, ], }, }, spacious:
{ on: { SET_DENSITY: { target:
'comfortable' } }, }, }, }, }, });

```

4.3 Onboarding Components

Component	File	Role	Key Props / Dependencies
OnboardingShell	OnboardingShell.tsx	Root container. Subscribes to OnboardingMachine via useMachine(). Renders current step through StepRenderer. Manages enter/exit animations.	None (top-level). Uses useOnboardingState() hook.
StepRenderer	StepRenderer.tsx	JSON-schema-driven step renderer. Reads onboarding.schema.json, maps each step's component field to a registered React	stepConfig: StepSchema, onEvent: (event) => void

Component	File	Role	Key Props / Dependencies
		component via ComponentRegistry.	
WelcomePane	WelcomePane.tsx	Animated glass-morphism welcome with Ionirix branding. Typewriter title effect. Emits NEXT or SKIP.	onNext: () => void, onSkip: () => void
CapabilitySelector	CapabilitySelector.tsx	Multi-select grid for platform capabilities (AI, Analytics, Spatial, etc.). Glass cards with selected-state glow. Reports selections upward.	capabilities: Capability[], selected: string[], onToggle: (id) => void
EnvironmentConfigurator	EnvironmentConfigurator.tsx	Theme, density, and motion preference controls. Live preview of selections. Segmented controls and sliders.	config: EnvironmentConfig, onChange: (config) => void
FlowSummary	FlowSummary.tsx	Animated summary of all selections before completion. Staggered reveal of choices with edit-in-place links. Emits COMPLETE.	context: OnboardingContext, onComplete: () => void, onEdit: (step) => void

4.4 Surface Components

Component	Architectural Role	Key Interface
AdaptiveSurface	Container that subscribes	zoneId: string,

Component	Architectural Role	Key Interface
	to reflow signals via useReflow(). When the ReflowEngine commits a new layout, AdaptiveSurface reads its zone's resolved dimensions and animates to the new position/size. Core building block for all emergent layout areas.	children: ReactNode, priority?: number, onReflow?: (resolved: ResolvedZone) => void
GlassCard	Glass-morphism card implementing the Ionix visual language: backdrop-filter: blur(24px), rgba(13,17,23,0.72) background, rgba(10,132,255,0.18) border, depth-aware inner glow. Supports depth levels 0-3 for layered stacking.	depth?: 0 1 2 3, glow?: boolean, interactive?: boolean, children: ReactNode
FloatingPanel	Absolutely positioned panel with spatial awareness. Queries SpatialAnalyzer to find open screen real estate, avoids overlap with other floating panels. Used for tooltips, contextual help, and overflow actions.	anchor: DOMRect, preferredPosition: 'top' 'bottom' 'left' 'right', avoidZones?: string[], children: ReactNode
ContextRibbon	Contextual action strip that emerges from the edge of the active zone. Appears when a behavior's evaluate() returns true. Contains action buttons relevant to the current context. Auto-dismisses on zone blur.	actions: RibbonAction[], position: 'top' 'bottom', visible: boolean

5. JSON-Driven Layout Schema

5.1 Schema Definition

Save as `schemas/layout.schema.json`. All layout surfaces in the Ionirix platform are defined by instances of this schema. The `ReflowEngine` and `LayoutResolver` consume these files at runtime.

```
{  "$schema": "https://json-schema.org/draft/2020-12/schema",  "$id": "https://ionirix.dev/schemas/layout.schema.json",  "title": "IonirixLayoutSchema",  "description": "Declarative layout definition for Ionirix Emergent-UI surfaces",  "type": "object",  "required": ["version", "surface"],  "properties": {    "version": {      "type": "string",      "pattern": "^\\d+\\.\\d+\\.\\d+$"    },    "surface": {      "type": "object",      "required": ["id", "type", "zones"],      "properties": {        "id": { "type": "string" },        "type": {          "enum": ["grid", "flex", "stack", "float", "emergent"],          "type": "string"        },        "zones": {          "type": "array",          "items": {            "type": "object",            "required": ["id", "component"],            "properties": {              "id": { "type": "string" },              "area": { "type": "string" },              "priority": {                "type": "integer",                "minimum": 0,                "maximum": 100              },              "responsive": {                "type": "object",                "properties": {                  "collapse": {                    "enum": ["hide", "minimize", "stack", "merge"],                    "type": "string"                  },                  "threshold": {                    "type": "integer",                    "description": "Viewport width in px below which collapse activates"                  },                  "reflow": {                    "type": "boolean",                    "description": "Whether to trigger ReflowEngine on collapse"                  }                },                "constraints": {                  "properties": {                    "minWidth": { "type": "string" },                    "maxWidth": { "type": "string" },                    "minHeight": { "type": "string" },                    "aspectRatio": { "type": "string" }                  },                  "type": "object"                },                "grid": {                  "properties": {                    "columns": { "type": "string" },                    "rows": { "type": "string" },                    "gap": { "type": "string" },                    "areas": {                    }                  },                  "type": "array"                },                "behaviors": {                  "type": "array",                  "items": {                    "type": "object",                    "required": ["id", "trigger", "action", "target"],                    "properties": {                      "id": { "type": "string" },                      "trigger": {                        "type": "string",                        "description": "Event type or pattern that activates this behavior"                      },                      "action": {                        "type": "string",                        "description": "Behavior action identifier from BehaviorRegistry"                      },                      "target": {                        "type": "string",                        "description": "Zone ID or '*' for all zones"                      },                      "params": {                        "type": "object",                        "additionalProperties": true                      },                      "transitions": {                        "type": "object",                        "properties": {                          "duration": {                            "type": "string",                            "default": "300ms"                          },                          "easing": {                            "type": "string",                            "default": "cubic-bezier(0.16, 1, 0.3, 1)"                          },                          "stagger": {                            "type": "string",                            "default": "cubic-bezier(0.16, 1, 0.3, 1)"                          }                        },                        "type": "object"                      },                      "type": "array"                    }                  },                  "type": "array"                },                "type": "string"              }            }          }        }      }    }  }
```

```
"type": "integer",          "default": 50,          "description": "Stagger
delay in ms between zone transitions"      }      }      }
```

5.2 Example: onboarding.schema.json

Save as `src/core/schema/defaults/onboarding.schema.json`. Defines a 5-step onboarding layout with adaptive zones that reflow based on user interaction.

```
{  "version": "1.0.0",  "surface": {    "id": "onboarding-root",    "type": "emergent",    "zones": [      {        "id": "progress-rail",        "component": "IonProgress",        "area": "rail",        "priority": 100,        "behavior": "contextual-elevation",        "responsive": { "collapse": "minimize", "threshold": 640, "reflow": false },        "constraints": { "minWidth": "48px", "maxWidth": "240px", "minHeight": "100vh" }      },      {        "id": "step-content",        "component": "StepRenderer",        "area": "main",        "priority": 95,        "behavior": "adaptive-reveal",        "responsive": { "collapse": "stack", "threshold": 640, "reflow": true },        "constraints": { "minWidth": "320px", "maxWidth": "720px" }      },      {        "id": "context-panel",        "component": "ContextRibbon",        "area": "context",        "priority": 60,        "behavior": "progressive-disclosure",        "responsive": { "collapse": "hide", "threshold": 768, "reflow": true },        "constraints": { "minWidth": "200px", "maxWidth": "320px" }      },      {        "id": "spatial-preview",        "component": "SpatialPreview",        "area": "preview",        "priority": 40,        "behavior": "spatial-collapse",        "responsive": { "collapse": "hide", "threshold": 1024, "reflow": true },        "constraints": { "minWidth": "280px", "aspectRatio": "16/9" }      },      {        "id": "action-bar",        "component": "ActionBar",        "area": "actions",        "priority": 90,        "responsive": { "collapse": "stack", "threshold": 480, "reflow": false },        "constraints": { "minHeight": "56px" }      }    ],    "grid": {      "columns": "auto 1fr 280px",      "rows": "1fr auto",      "gap": "16px",      "areas": [        "rail main context",        "rail actions actions"      ]    },    "behaviors": [      {        "id": "reveal-context-on-selection",        "trigger": "SELECT_CAPABILITY",        "action": "adaptive-reveal",        "target": "context-panel",        "params": { "delay": 300, "direction": "right" }      },      {        "id": "collapse-preview-on-narrow",        "trigger": "VIEWPORT_RESIZE",        "action": "spatial-collapse",        "target": "spatial-preview",        "params": { "threshold": 1024, "animation": "slide-out" }      },      {        "id": "elevate-active-step",        "trigger": "INTERACTION",        "action": "contextual-elevation",        "target": "step-content",        "params": { "zIndexDelta": 10, "scaleFactor": 1.01, "glowColor": "#0A84FF" }      },      {        "id": "disclose-advanced-options",        "trigger": "CONFIGURE_ENV",        "action": "progressive-disclosure",        "target": "step-content",        "params": { "depthThreshold": 3, "revealSelector": ".advanced-options" }      }    ],    "transitions": {      "duration": "500ms",      "easing": "cubic-bezier(0.16, 1, 0.3, 1)",      "stagger": 60    }  }
```

6. Event-Driven Reflow Engine

6.1 EventBus

Typed pub/sub system using discriminated union of event types. Zero external dependencies. All inter-layer communication routes through EventBus.

```
// src/core/events/EventTypes.ts export type IonirixEvent = |
{
  type: 'INTERACTION';
  payload: {
    zoneId: string;
    action: 'click' | 'hover' | 'focus' | 'scroll' | 'drag';
    position: { x: number; y: number };
    timestamp: number;
  };
} | {
  type: 'REFLOW_REQUEST';
  payload: {
    trigger: 'viewport' | 'interaction' | 'behavior' | 'state-change';
    source: string;
    priority: 'immediate' | 'deferred' | 'lazy';
  };
} | {
  type: 'LAYOUT_CHANGE';
  payload: {
    previousLayout: string; // serialized layout hash
    nextLayout: string;
    mutations: Array<{
      zoneId: string; property: string; value: string | number }>;
    timestamp: number;
  };
} | {
  type: 'BEHAVIOR_TRIGGER';
  payload: {
    behaviorId: string;
    targetZone: string;
    params: Record<string, unknown>;
  };
} | {
  type: 'STATE_TRANSITION';
  payload: {
    machine: 'onboarding' | 'ui';
    from: string; to: string;
    event: string;
  };
} | {
  type: 'SPATIAL_UPDATE';
  payload: {
    collisions: Array<{ zoneA: string; zoneB: string }>;
    balance: { horizontal: number; vertical: number };
  };
}; export type IonirixEventType =
IonirixEvent['type']; // Extract payload type for a given event type export
type EventPayload<T extends IonirixEventType> = Extract< IonirixEvent,
{ type: T } >['payload'];
// src/core/events/EventBus.ts import type { IonirixEvent, IonirixEventType,
EventPayload } from './EventTypes'; type Handler<T extends IonirixEventType>
= (payload: EventPayload<T>) => void; interface Subscription
{ unsubscribe: () => void; } export class EventBus { private handlers =
new Map<IonirixEventType, Set<Handler<any>>>(); private history:
IonirixEvent[] = []; private maxHistory = 1000; /** Subscribe to a typed
event */ on<T extends IonirixEventType>( type: T, handler:
Handler<T> ): Subscription { if (!this.handlers.has(type))
{ this.handlers.set(type, new Set()); }
this.handlers.get(type)!.add(handler); return { unsubscribe: () =>
this.off(type, handler), }; } /** Unsubscribe a handler */ off<T
extends IonirixEventType>(type: T, handler: Handler<T>): void
{ this.handlers.get(type)?.delete(handler); } /** Subscribe to a
single occurrence */ once<T extends IonirixEventType>( type: T,
handler: Handler<T> ): Subscription { const wrapper: Handler<T> =
(payload) => { this.off(type, wrapper); handler(payload); };
return this.on(type, wrapper); } /** Emit a typed event to all
subscribers */ emit<T extends IonirixEventType>( type: T, payload:
EventPayload<T> ): void { const event = { type, payload } as
IonirixEvent; // Record in history (ring buffer)
this.history.push(event); if (this.history.length > this.maxHistory) {
this.history.shift(); } // Dispatch to handlers const handlers =
this.handlers.get(type); if (handlers) { handlers.forEach((handler)
=> { try { handler(payload); } catch (error) {
console.error(`[EventBus] Error in ${type} handler:`, error); }
}); } } /** Get event history (readonly) */ getHistory():
ReadonlyArray<IonirixEvent> { return this.history; } /** Clear all
```

```
subscriptions */ dispose(): void { this.handlers.clear();
this.history = []; } } // Singleton instance export const EventBus = new
EventBus();
```

6.2 ReflowHandler

Listens for `REFLOW_REQUEST` events, invokes the `ReflowEngine`, and emits `LAYOUT_CHANGE` with the resolved layout diff. Acts as the bridge between the event layer and the engine layer.

```
// src/core/events/handlers/ReflowHandler.ts import { EventBus } from
'../EventBus'; import { ReflowEngine } from '../../engine/ReflowEngine';
import { BehaviorRegistry } from '../../registry/BehaviorRegistry'; import
type { EventPayload } from '../EventTypes'; import type { ReflowTrigger }
from '@types'; export class ReflowHandler { private debounceTimer:
ReturnType<typeof setTimeout> | null = null; private debounceMs = 16; // ~1
frame at 60fps constructor( private bus: EventBus, private engine:
ReflowEngine, private registry: BehaviorRegistry ) {
this.bus.on('REFLOW_REQUEST', this.handleReflowRequest.bind(this)); }
private handleReflowRequest( payload: EventPayload<'REFLOW_REQUEST'> ):
void { // Immediate priority bypasses debounce if (payload.priority
=== 'immediate') { this.executeReflow(payload); return; }
// Debounce deferred and lazy requests if (this.debounceTimer)
clearTimeout(this.debounceTimer); const delay = payload.priority ===
'lazy' ? 100 : this.debounceMs; this.debounceTimer = setTimeout(() => {
this.executeReflow(payload); }, delay); } private executeReflow(
payload: EventPayload<'REFLOW_REQUEST'> ): void { const trigger:
ReflowTrigger = { source: payload.source, type: payload.trigger,
timestamp: Date.now(), }; // 1. Request reflow from engine const
result = this.engine.requestReflow(trigger); // 2. Apply active
behaviors const activeBehaviors = this.registry.getActive(); const
finalLayout = this.engine.applyBehaviors( result.layout,
activeBehaviors ); // 3. Commit the layout
this.engine.commit(finalLayout); // 4. Emit layout change event
this.bus.emit('LAYOUT_CHANGE', { previousLayout:
this.hashLayout(result.layout), nextLayout:
this.hashLayout(finalLayout), mutations: result.mutations.map((m) => ({
zoneId: m.zoneId, property: m.property, value: m.nextValue,
})), timestamp: Date.now(), }); } private hashLayout(layout:
any): string { return JSON.stringify(layout).slice(0, 64); }
dispose(): void { if (this.debounceTimer)
clearTimeout(this.debounceTimer); } }
```

7. Adaptive Behavior Registry

7.1 Behavior Interface & Registry

```
// src/types/behavior.types.ts export interface BehaviorContext {
viewport: { width: number; height: number }; activeZones: string[];
focusedZone: string | null; interactionHistory: Array<{ event: string;
timestamp: number }>; userPreferences: Record<string, unknown>;
currentState: string; } export interface SurfaceUpdate { zoneId: string;
```

```

mutations: Array<{    property: string;    value: string | number |
boolean;    }>;    animate: boolean;    duration?: number; } export interface
Behavior {    id: string;    name: string;    description: string;    priority:
number;                                // 0-100, higher = evaluated first
trigger: string;                                // Event pattern (e.g.,
'INTERACTION')    evaluate: (context: BehaviorContext) => boolean;    execute:
(    zoneId: string,    params: Record<string, unknown>    ) =>
SurfaceUpdate;    cleanup?: (zoneId: string) => void; }
// src/core/registry/BehaviorRegistry.ts    import type { Behavior,
BehaviorContext } from '@types/behavior.types';    export class
BehaviorRegistry {    private static instance: BehaviorRegistry;    private
behaviors = new Map<string, Behavior>();    private activeSet = new
Set<string>();    private constructor() {}    static getInstance():
BehaviorRegistry {    if (!BehaviorRegistry.instance)
{    BehaviorRegistry.instance = new BehaviorRegistry();    }    return
BehaviorRegistry.instance;    }    /** Register a behavior implementation */
register(id: string, behavior: Behavior): void {    if
(this.behaviors.has(id)) {    console.warn(`[BehaviorRegistry] Overwriting
behavior: ${id}`);    }    this.behaviors.set(id, behavior);    }    /**
Resolve a behavior by ID */    resolve(id: string): Behavior | undefined {
return this.behaviors.get(id);    }    /** Get all currently active behaviors,
sorted by priority (desc) */    getActive(): Behavior[] {    return
Array.from(this.activeSet)    .map((id) =>
this.behaviors.get(id))    .filter((b): b is Behavior => b !== undefined)
.sort((a, b) => b.priority - a.priority);    }    /** Evaluate all registered
behaviors against current context.    * Activates those whose evaluate()
returns true. */    evaluate(context: BehaviorContext): Behavior[] {    const
activated: Behavior[] = [];    for (const [id, behavior] of this.behaviors)
{    const shouldActivate = behavior.evaluate(context);    if
(shouldActivate && !this.activeSet.has(id)) {    this.activeSet.add(id);
activated.push(behavior);    } else if (!shouldActivate &&
this.activeSet.has(id)) {    this.activeSet.delete(id);
behavior.cleanup?.(id);    }    }    return activated;    }    /** Get
all registered behavior IDs */    getRegisteredIds(): string[] {    return
Array.from(this.behaviors.keys());    }    /** Reset all active behaviors */
reset(): void {    for (const id of this.activeSet)
{    this.behaviors.get(id)?.cleanup?.(id);    }
this.activeSet.clear();    } }

```

7.2 Built-in Behaviors

7.2.1 AdaptiveReveal

Progressive disclosure based on user engagement depth. Zones reveal additional content as the user demonstrates sustained interaction with a particular area.

```

// src/core/registry/behaviors/AdaptiveReveal.ts    import type { Behavior,
BehaviorContext, SurfaceUpdate } from '@types/behavior.types';    export const
AdaptiveReveal: Behavior = {    id: 'adaptive-reveal',    name: 'Adaptive
Reveal',    description: 'Reveals content progressively based on user
engagement depth',    priority: 80,    trigger: 'INTERACTION',
evaluate(context: BehaviorContext): boolean {    // Activate when user has
interacted with focused zone 3+ times    if (!context.focusedZone) return
false;    const zoneInteractions =
context.interactionHistory.filter(    (h) =>
h.event.includes(context.focusedZone!)    );    return

```

```

zoneInteractions.length >= 3; }, execute(zoneId: string, params:
Record<string, unknown>): SurfaceUpdate { const delay = (params.delay as
number) ?? 300; const direction = (params.direction as string) ?? 'down';
return { zoneId, mutations: [ { property: 'data-reveal-
state', value: 'revealed' }, { property: 'data-reveal-direction',
value: direction }, ], animate: true, duration:
delay, }; }, cleanup(zoneId: string): void { // Reset reveal
state when behavior deactivates }, };

```

7.2.2 SpatialCollapse

Collapses low-priority zones when viewport shrinks below their configured threshold. Unlike simple responsive breakpoints, SpatialCollapse considers zone priority, current interaction focus, and neighboring zone relationships.

```

// src/core/registry/behaviors/SpatialCollapse.ts import type { Behavior,
BehaviorContext, SurfaceUpdate } from '@types/behavior.types'; export const
SpatialCollapse: Behavior = { id: 'spatial-collapse', name: 'Spatial
Collapse', description: 'Collapses low-priority zones based on viewport and
spatial context', priority: 90, trigger: 'VIEWPORT_RESIZE',
evaluate(context: BehaviorContext): boolean { // Activate when viewport
is below comfortable threshold for current zone count const zoneCount =
context.activeZones.length; const minWidthPerZone = 280; return
context.viewport.width < zoneCount * minWidthPerZone; }, execute(zoneId:
string, params: Record<string, unknown>): SurfaceUpdate { const threshold
= (params.threshold as number) ?? 768; const animation =
(params.animation as string) ?? 'fade-out'; return { zoneId,
mutations: [ { property: 'visibility', value: 'collapsed' },
{ property: 'data-collapse-animation', value: animation }, ],
animate: true, duration: 300, }; }, };

```

7.2.3 ContextualElevation

Elevates the active zone (z-index, subtle scale, glow) based on interaction focus. Creates depth hierarchy that communicates what the user is currently engaged with.

```

// src/core/registry/behaviors/ContextualElevation.ts import type
{ Behavior, BehaviorContext, SurfaceUpdate } from '@types/behavior.types';
export const ContextualElevation: Behavior = { id: 'contextual-elevation',
name: 'Contextual Elevation', description: 'Elevates the focused zone with
z-index, scale, and glow', priority: 70, trigger: 'INTERACTION',
evaluate(context: BehaviorContext): boolean { return
context.focusedZone !== null; }, execute(zoneId: string, params:
Record<string, unknown>): SurfaceUpdate { const zIndexDelta =
(params.zIndexDelta as number) ?? 10; const scaleFactor =
(params.scaleFactor as number) ?? 1.02; const glowColor =
(params.glowColor as string) ?? '#0A84FF'; return { zoneId,
mutations: [ { property: 'z-index-delta', value: zIndexDelta },
{ property: 'scale', value: scaleFactor }, { property: 'glow-color',
value: glowColor }, { property: 'data-elevated', value: true }, ],
animate: true, duration: 200, }; }, cleanup(zoneId:
string): void { // Remove elevation when zone loses focus }, };

```

7.2.4 ProgressiveDisclosure

Reveals complexity layers as the user demonstrates mastery. Tracks interaction depth (number of unique actions performed) and progressively unhides advanced UI elements.

```
// src/core/registry/behaviors/ProgressiveDisclosure.ts import type
{ Behavior, BehaviorContext, SurfaceUpdate } from '@types/behavior.types';
export const ProgressiveDisclosure: Behavior = { id: 'progressive-
disclosure', name: 'Progressive Disclosure', description: 'Reveals
advanced UI elements as user demonstrates mastery', priority: 60,
trigger: 'STATE_TRANSITION', evaluate(context: BehaviorContext): boolean {
// Activate when user has performed enough unique action types const
uniqueActions = new Set( context.interactionHistory.map((h) => h.event)
); const depthThreshold = 3; return uniqueActions.size >=
depthThreshold; }, execute(zoneId: string, params: Record<string,
unknown>): SurfaceUpdate { const revealSelector = (params.revealSelector
as string) ?? '.advanced-options'; const depthThreshold =
(params.depthThreshold as number) ?? 3; return { zoneId,
mutations: [ { property: 'data-disclosure-level', value:
depthThreshold }, { property: 'data-reveal-selector', value:
revealSelector }, { property: 'data-disclosed', value:
true }, ], animate: true, duration: 400, }; }, };
```

8. Onboarding Flow Implementation Plan

8.1 Phased Roadmap

Phase	Timeline	Deliverables	Dependencies
Phase 1: Foundation	Week 1-2	Project scaffold, design tokens, primitive components, OnboardingMachine, basic EventBus	None
Phase 2: Schema Engine	Week 3-4	LayoutSchema JSON format, LayoutResolver, StepRenderer, onboarding.schema.json, OnboardingShell wiring	Phase 1

Phase	Timeline	Deliverables	Dependencies
Phase 3: Spatial Intelligence	Week 5–6	SpatialAnalyzer, ReflowEngine with constraints, AdaptiveSurface, EmergentGrid, ZoneManager	Phase 1, Phase 2
Phase 4: Behavior System	Week 7–8	BehaviorRegistry, 4 core behaviors, EventBus wiring, ContextRibbon, interaction history	Phase 1, Phase 3
Phase 5: Polish & Emergence	Week 9–10	Glass-morphism pass, Framer Motion animations, spatial calibration, persistence, tests, docs	Phase 2, Phase 3, Phase 4

8.2 Phase Details

Phase 1: Foundation (Week 1-2)

1. Scaffold project: `npm create vite@latest ionirix-emergent-ui -- --template react-ts`
2. Install core dependencies (see Section 10)
3. Configure `tsconfig.json` with strict mode, path aliases (`@/` → `src/`)
4. Configure `tailwind.config.ts` with Ionirix design tokens as theme extensions
5. Create `src/styles/tokens.css` with all CSS custom properties (see Section 9)
6. Build primitive components: `IonButton`, `IonInput`, `IonToggle`, `IonProgress`, `IonTooltip`
7. Implement `onboardingMachine` with XState v5 `setup()` API
8. Implement basic `EventBus` with typed pub/sub
9. Create type definition files in `src/types/`

Milestone Check

At the end of Phase 1, you should be able to: start the dev server, render primitive components, and step through the OnboardingMachine in the Stately VS Code extension visualizer.

Phase 2: Schema Engine (Week 3-4)

10. Define `schemas/layout.schema.json` (JSON Schema for layout definitions)
11. Implement `LayoutSchema.ts` — TypeScript types matching the JSON schema
12. Build `validators.ts` using Zod schemas that mirror the JSON Schema
13. Implement `LayoutResolver` — resolve JSON to concrete grid definitions
14. Build `StepRenderer` — reads step config, maps `component` field to React components
15. Build `ComponentRegistry` — maps string component names to React component references
16. Create `onboarding.schema.json` with 5 steps and adaptive zones
17. Wire `OnboardingShell` to state machine with `@xstate/react`'s `useMachine()`
18. Build `WelcomePane`, `CapabilitySelector`, `EnvironmentConfigurator`, `FlowSummary`

Phase 3: Spatial Intelligence (Week 5-6)

19. Build `SpatialAnalyzer` — DOM rect monitoring, overlap detection, balance assessment
20. Implement `ReflowEngine` — schema-driven layout resolution with constraint satisfaction
21. Create `AdaptiveSurface` — subscribes to reflow signals, animates dimension changes
22. Build `EmergentGrid` — CSS Grid container managed by `LayoutResolver` output

- 23. Implement `ZoneManager` — coordinates multiple zones, manages z-index stacking
- 24. Create `useReflow()` hook — subscribes component to `ReflowEngine` output for its zone
- 25. Create `useSpatialContext()` hook — provides `SpatialAnalyzer` data to components
- 26. Wire `ReflowHandler` to `EventBus`

Phase 4: Behavior System (Week 7-8)

- 27. Build `BehaviorRegistry` singleton with register/resolve/evaluate API
- 28. Implement 4 core behaviors: `AdaptiveReveal`, `SpatialCollapse`, `ContextualElevation`, `ProgressiveDisclosure`
- 29. Wire behavior evaluation to `EventBus` event cycle
- 30. Build `ContextRibbon` — behavior-driven contextual action strip
- 31. Build `FloatingPanel` with `SpatialAnalyzer`-driven positioning
- 32. Implement `useBehavior()` hook — components query active behaviors for their zone
- 33. Add interaction history tracking to `EventBus` (ring buffer, configurable depth)
- 34. Create `GlassCard` with depth levels 0-3

Phase 5: Polish & Emergence (Week 9-10)

- 35. Glass-morphism styling pass: apply `glass.css` tokens across all surfaces
- 36. Animation system: integrate `Framer Motion` for layout animations and zone transitions
- 37. Build spatial calibration step: interactive zone drag/resize for user layout preference discovery
- 38. Persistence layer: `LocalStorage` adapter for onboarding state, user prefs, interaction history

39. Write test suite: OnboardingMachine state transitions, ReflowEngine constraint resolution, BehaviorRegistry evaluation
40. Write documentation: ARCHITECTURE.md, BEHAVIOR_CATALOG.md, SCHEMA_REFERENCE.md
41. Final integration test: complete onboarding flow end-to-end

8.3 Dependency Graph

The following table shows the dependency relationships. A phase can begin only when all its dependencies have completed.

Phase	Depends On	Blocks	Can Parallel With
Phase 1: Foundation	—	Phase 2, 3, 4, 5	—
Phase 2: Schema Engine	Phase 1	Phase 5	Phase 3 (partial overlap OK: Phase 2 delivers LayoutResolver first)
Phase 3: Spatial Intelligence	Phase 1, Phase 2 (LayoutResolver)	Phase 4, 5	Phase 2 (latter half)
Phase 4: Behavior System	Phase 1 (EventBus), Phase 3 (SpatialAnalyzer, ReflowEngine)	Phase 5	—
Phase 5: Polish & Emergence	Phase 2, 3, 4	—	—

Critical Path

The critical path runs: **Phase 1 → Phase 3 → Phase 4 → Phase 5**. Phase 2 can partially overlap with Phase 3 since the LayoutResolver (Phase 2) is needed by the ReflowEngine (Phase 3), but StepRenderer and onboarding components can be built in parallel with SpatialAnalyzer work.

9. Design Tokens Reference

Save as `src/styles/tokens.css`. These tokens define the entire Ionirix visual language. All components reference these values; no hardcoded color or spacing values anywhere else in the codebase.

```
/* src/styles/tokens.css – Ionirix Design Token System */ :root { /* —
Brand Palette _____ */ --ion-blue: #0A84FF;
--spectral-cyan: #00E5FF; --quantum-white: #F0F4FF; --amber: #FFB800;
--pine-black: #0D1117; /* — Semantic Colors
_____ */ --color-primary: #0A84FF; --color-
secondary: #00E5FF; --color-success: #30D158; --color-warning: #FFB800;
--color-error: #FF453A; --color-info: #64D2FF; /* — Surface System
_____ */ --glass-bg: rgba(13, 17, 23, 0.72);
--glass-bg-hover: rgba(13, 17, 23, 0.82); --glass-border: rgba(10, 132,
255, 0.18); --glass-border-active: rgba(10, 132, 255, 0.36); --glass-
blur: 24px; --glass-blur-heavy: 40px; --surface-0: #0D1117; --surface-
1: #161B22; --surface-2: #21262D; --surface-3: #30363D; --surface-
radius: 16px; --surface-radius-sm: 8px; --surface-radius-lg: 24px; /*
— Typography _____ */ --font-display:
'Inter', system-ui, sans-serif; --font-body: 'Inter', system-ui, sans-
serif; --font-mono: 'JetBrains Mono', 'Fira Code', monospace; --text-xs:
0.75rem; /* 12px */ --text-sm: 0.875rem; /* 14px */ --text-base:
1rem; /* 16px */ --text-lg: 1.125rem; /* 18px */ --text-xl:
1.25rem; /* 20px */ --text-2xl: 1.5rem; /* 24px */ --text-3xl:
1.875rem; /* 30px */ --text-4xl: 2.25rem; /* 36px */ --leading-tight:
1.25; --leading-normal: 1.5; --leading-relaxed: 1.75; /* — Spacing
Scale _____ */ --space-px: 1px; --space-
0: 0px; --space-xs: 4px; --space-sm: 8px; --space-md: 16px; --space-
lg: 24px; --space-xl: 32px; --space-2xl: 48px; --space-3xl: 64px; --
space-4xl: 96px; /* — Z-Index Scale _____ */
*/ --z-base: 0; --z-raised: 10; --z-overlay: 100; --z-modal: 200;
--z-toast: 300; --z-tooltip: 400; /* — Transitions
_____ */ --ease-emergence: cubic-
bezier(0.16, 1, 0.3, 1); --ease-settle: cubic-bezier(0.33, 1, 0.68, 1);
--ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1); --duration-instant:
50ms; --duration-fast: 150ms; --duration-normal: 300ms; --duration-
slow: 500ms; --duration-emerge: 800ms; --duration-cinematic:
1200ms; /* — Shadows (Depth Levels) _____ */ --
shadow-0: none; --shadow-1: 0 1px 3px rgba(0, 0, 0, 0.5); --shadow-2: 0
4px 12px rgba(0, 0, 0, 0.6); --shadow-3: 0 8px 32px rgba(0, 0, 0,
0.7); /* — Glow Effects _____ */ --
glow-blue: 0 0 20px rgba(10, 132, 255, 0.3); --glow-cyan: 0 0 20px rgba(0,
229, 255, 0.3); --glow-amber: 0 0 20px rgba(255, 184, 0, 0.3); }
```

10. Package Dependencies

10.1 Production Dependencies

Package	Version	Purpose
react	^19.0.0	UI library — component rendering
react-dom	^19.0.0	DOM rendering target
xstate	^5.18.0	State machine engine (OnboardingMachine, UIStateMachine)
@xstate/react	^5.0.0	React bindings for XState — useMachine(), useActor()
framer-motion	^12.0.0	Layout animations, zone transitions, spring physics
zod	^3.24.0	Runtime schema validation for LayoutSchema JSON

10.2 Development Dependencies

Package	Version	Purpose
typescript	^5.7.0	Type system — strict mode enabled
vite	^6.0.0	Build tool — dev server, HMR, production bundling
@vitejs/plugin-react	^4.4.0	React Fast Refresh for Vite
tailwindcss	^4.0.0	Utility-first CSS — extended with Ionirix design tokens
@tailwindcss/forms	^0.5.10	Form element resets and styling
vitest	^3.0.0	Test runner — Vite-

Package	Version	Purpose
		native, Jest-compatible API
@testing-library/react	^16.0.0	Component testing utilities
@testing-library/jest-dom	^6.6.0	DOM assertion matchers
eslint	^9.0.0	Linting — flat config format
@eslint/js	^9.0.0	ESLint core rules
typescript-eslint	^8.0.0	TypeScript-aware ESLint rules
prettier	^3.4.0	Code formatter — single quotes, trailing commas
@types/react	^19.0.0	React type definitions
@types/react-dom	^19.0.0	React DOM type definitions

10.3 Install Command

```
# Production npm install react react-dom xstate @xstate/react framer-motion zod
# Development npm install -D typescript vite @vitejs/plugin-react tailwindcss \
  @tailwindcss/forms vitest @testing-library/react \
  @testing-library/jest-dom eslint @eslint/js typescript-eslint \
  prettier @types/react @types/react-dom
```

10.4 Configuration Files

tsconfig.json

```
{  "compilerOptions": {    "target": "ES2022",    "lib": ["ES2022", "DOM", "DOM.Iterable"],    "module": "ESNext",    "moduleResolution": "bundler",    "jsx": "react-jsx",    "strict": true,    "noUncheckedIndexedAccess": true,    "noUnusedLocals": true,    "noUnusedParameters": true,    "exactOptionalPropertyTypes": true,    "forceConsistentCasingInFileNames": true,    "skipLibCheck": true,    "resolveJsonModule": true,    "isolatedModules": true,    "paths": {      "@/*": ["../src/*"]    },    "baseUrl": ".",    "include": ["src", "tests"],    "exclude": ["node_modules", "dist"]  }
```

vite.config.ts

```
import { defineConfig } from 'vite'; import react from '@vitejs/plugin-react'; import { resolve } from 'path'; export default
```

```
defineConfig({  plugins: [react()],  resolve: {    alias: {      '@': resolve(__dirname, 'src'),    },    server: {      port: 5173,      open: true,    },    build: {      target: 'es2022',      sourcemap: true,    },  });
```

tailwind.config.ts

```
import type { Config } from 'tailwindcss'; export default { content:
['./index.html', './src/**/*.ts,tsx'], theme: { extend:
{ colors: { ion: { blue: '#0A84FF', cyan:
'#00E5FF', white: '#F0F4FF', amber: '#FFB800',
black: '#0D1117', }, surface: { 0: '#0D1117',
1: '#161B22', 2: '#21262D', 3: '#30363D', },
}, fontFamily: { display: ['Inter', 'system-ui', 'sans-serif'],
mono: ['JetBrains Mono', 'Fira Code', 'monospace'], },
transitionTimingFunction: { emergence: 'cubic-bezier(0.16, 1, 0.3,
1)', settle: 'cubic-bezier(0.33, 1, 0.68, 1)', spring:
'cubic-bezier(0.34, 1.56, 0.64, 1)', }, animation:
{ emerge: 'emerge 800ms cubic-bezier(0.16, 1, 0.3, 1)', glow:
'glow 2s ease-in-out infinite alternate', }, keyframes: {
emerge: { '0%': { opacity: '0', transform: 'translateY(12px)
scale(0.96)' }, '100%': { opacity: '1', transform: 'translateY(0)
scale(1)' }, }, glow: { '0%': { boxShadow: '0 0
12px rgba(10, 132, 255, 0.2)' }, '100%': { boxShadow: '0 0 24px
rgba(10, 132, 255, 0.4)' }, }, }, }, plugins:
[require('@tailwindcss/forms')], } satisfies Config;
```

Appendix: Quick Reference

A.1 State Machine Event Reference

Event	Source State(s)	Target State	Guard	Side Effects
NEXT	All except complete	Next sequential state	State-dependent (e.g., hasProfile)	recordInteraction, persistState
BACK	All except idle, welcome	Previous state	canGoBack	decrementStep, recordInteraction
SKIP	welcome	capabilitySelection	None	recordInteraction
UPDATE_PROFILE	profiling	(self)	None	updateProfile, recordInteraction
SELECT_CAPABILITY	capabilitySelection	(self)	None	addCapability, recordInteraction

Event	Source State(s)	Target State	Guard	Side Effects
CONFIGURE_ENV	environmentSetup	(self)	None	setEnvironment, recordInteraction
CALIBRATE	spatialCalibration	(self)	None	recordInteraction
COMPLETE	summary	complete (final)	None	recordInteraction, persistState

A.2 Behavior Priority Matrix

Behavior	Priority	Trigger Event	Activation Condition
SpatialCollapse	90	VIEWPORT_RESIZE	Viewport width < (zoneCount × 280px)
AdaptiveReveal	80	INTERACTION	≥3 interactions with focused zone
ContextualElevation	70	INTERACTION	Any zone is focused
ProgressiveDisclosure	60	STATE_TRANSITION	≥3 unique action types in history

A.3 Ionirix Color Reference

Token Name	Hex Value	Usage
Ion Blue	#0A84FF	Primary interactive elements, focus rings, active states, glow effects
Spectral Cyan	#00E5FF	Secondary accents, data visualizations, hover highlights
Quantum White	#F0F4FF	Primary text on dark surfaces, headings

Token Name	Hex Value	Usage
Amber	#FFB800	Warnings, notifications, attention markers
Pine Black	#0D1117	Background surfaces, base layer, deep shadows

A.4 File → Section Cross-Reference

File Path	Defined In	Purpose
src/core/engine/ReflowEngine.ts	Section 4.1.1, 6.2	Layout recalculation orchestrator
src/core/engine/LayoutResolver.ts	Section 4.1.2	JSON schema → concrete CSS grid
src/core/engine/SpatialAnalyzer.ts	Section 4.1.3	Zone relationship monitor
src/core/state/OnboardingMachine.ts	Section 4.2.1	XState v5 onboarding flow machine
src/core/state/UIStateMachine.ts	Section 4.2.2	Parallel UI mode/density machine
src/core/events/EventBus.ts	Section 6.1	Typed pub/sub system
src/core/events/EventTypes.ts	Section 6.1	Discriminated union event definitions
src/core/events/handlers/ReflowHandler.ts	Section 6.2	REFLOW_REQUEST → LAYOUT_CHANGE bridge
src/core/registry/BehaviorRegistry.ts	Section 7.1	Singleton behavior manager
src/core/registry/behaviors/*.ts	Section 7.2	4 core behavior implementations
schemas/layout.schema.json	Section 5.1	JSON Schema for layout definitions
src/core/schema/defaults/onboarding.schema.json	Section 5.2	5-step onboarding layout config
src/styles/tokens.css	Section 9	Complete design token system

Next Steps

Open the `.code-workspace` file in VS Code, install the recommended extensions, run `npm install`, and begin with Phase 1. The `OnboardingMachine` is implementation-ready — paste the XState configuration from Section 4.2.1 into `src/core/state/OnboardingMachine.ts` and visualize it immediately with the Stately VS Code extension.